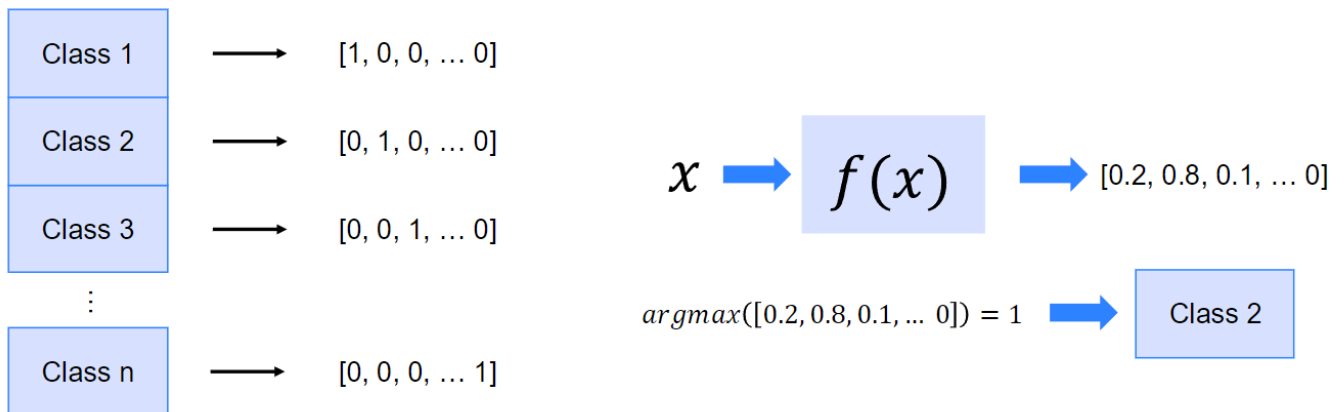


Session 03 - Intro to Neural Networks

One-Hot Encoding

One-hot encoding is a method for transforming categorical data into a numerical format that machine learning models can process. This is used for Classification, so no continuous values but clearly separated classes of data!



Multiple classes could be "Class 1," "Class 2," "Class 3," ... "Class n," each class is represented by a vector where a single position is "1" and all others are "0".

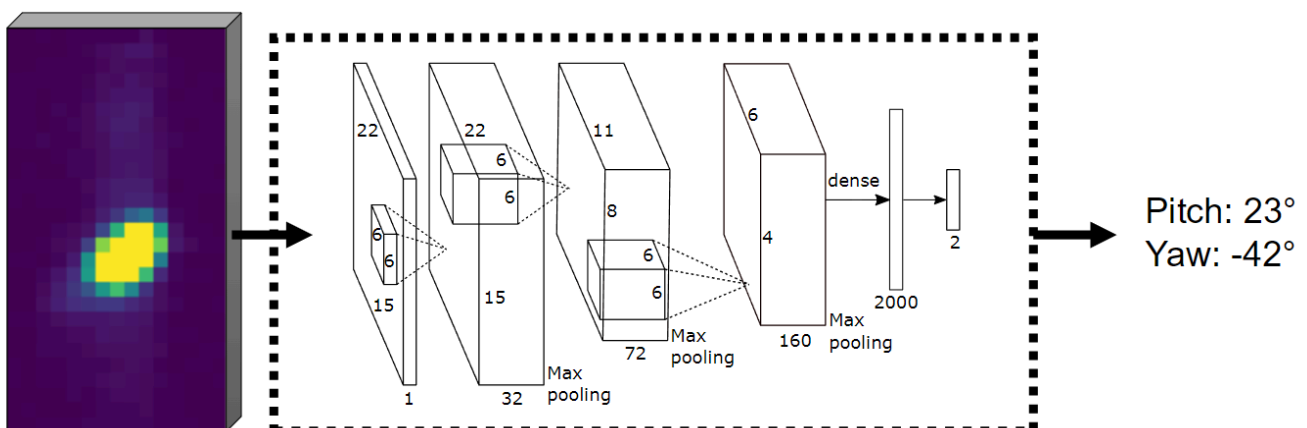
⚡ Classification has still a probability

While these vectors are not probabilities, it is helpful to conceptualize them as representing the likelihood of belonging to a specific class. When a model outputs a vector like:

[0.2, 0.8, 0.1, ..., 0], the argmax function is used to identify the index of the highest value, thereby determining the predicted class!

$\text{argmax}([0.2, 0.8, 0.1, \dots 0]) = 1$, which corresponds to Class 2.

Machine Learning to Enhance Sensing



Convolutional Neural Networks (CNNs) can process an input, pass it through various layers including convolutional, pooling, and dense layers, to produce a specific output, such as pitch and yaw angles of a finger on a touchscreen just based on the point the touchscreen is registering input.

Neural Network Structure and Implementation

Neural networks are built by stacking layers, each performing a specific transformation on the input data.

TensorFlow Syntax for a 2-Layer Model

```
# Creating a Sequential model - a linear stack of layers
model = tf.keras.Sequential()

# Adding an input layer that expects vectors of size 400 (e.g., flattened input)
model.add(tf.keras.layers.InputLayer((400,), name="InputLayer"))

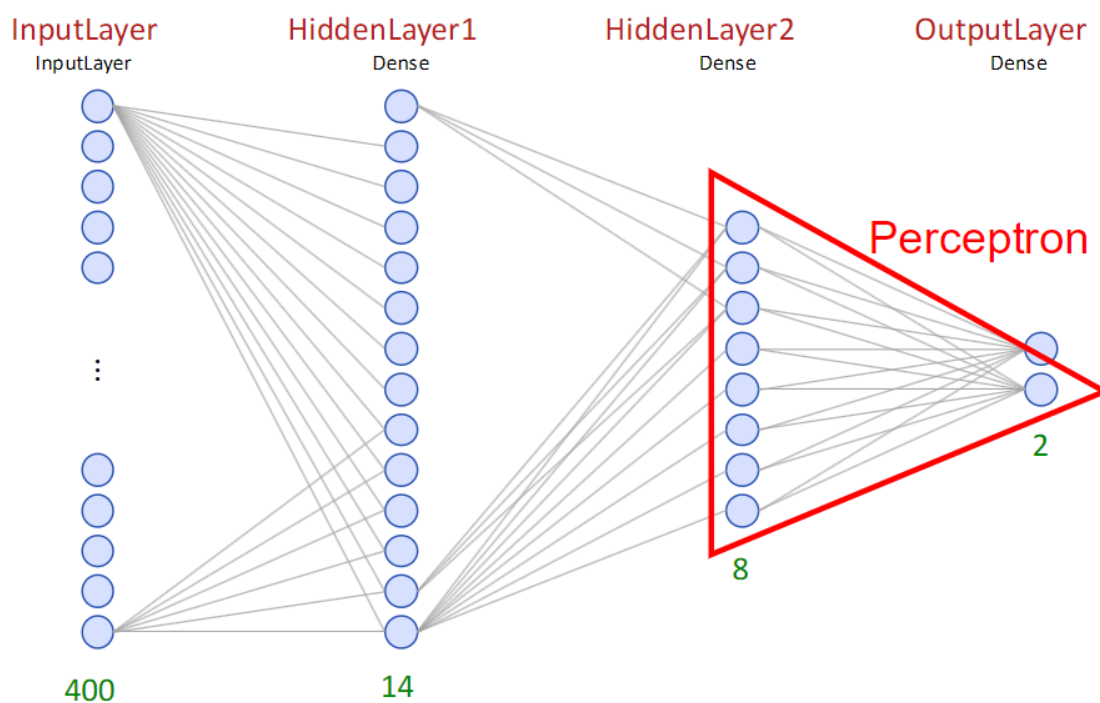
# Adding a dense (fully connected) hidden layer with 14 neurons and ReLU activation
model.add(tf.keras.layers.Dense(14, name="HiddenLayer1", activation='relu'))

# Adding another dense hidden layer with 8 neurons and ReLU activation
model.add(tf.keras.layers.Dense(8, name="HiddenLayer2", activation='relu'))

# Adding the final output layer with 2 neurons and softmax activation
# This is used for binary classification (2 classes, with probabilities summing to 1 - done by softmax)
model.add(tf.keras.layers.Dense(2, name="OutputLayer", activation='softmax'))
```

The defined neural network in the code above consists of interconnected layers, with each connection having an associated weight.

This network has an input layer, two hidden layers, and an output layer and looks like this:

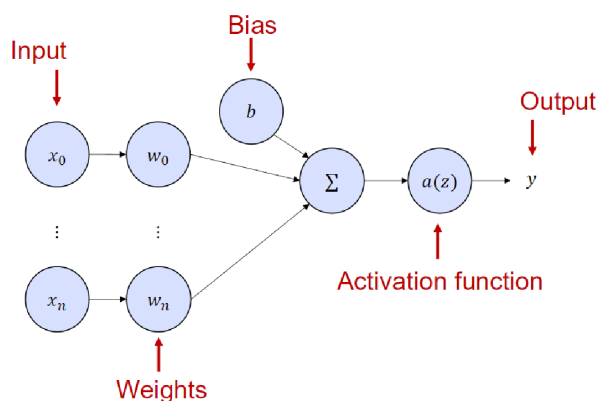


- **Input Layer:** Receives the raw data, here with 400 units
e.g., word counts in an email or email metadata
- **Hidden Layer 1 (Dense):** Contains 14 neurons.
Learns patterns from the input, like common spammy phrases
- **Hidden Layer 2 (Dense):** Contains 8 neurons.
Refines the patterns into higher-level clues.
- **Output Layer (Dense):** Produces the final output, here with 2 neurons.
Decision: spam or not spam (as probabilities, that add up to 1).

Each neuron in the hidden and output layers processes its inputs, applies an activation function, and passes the result to the next layer.

Single-Layer Perceptron

The Perceptron, proposed by Frank Rosenblatt in 1958



$$f(x) = f\left(\begin{bmatrix} x_0 \\ \vdots \\ x_n \end{bmatrix}\right) = a(x \cdot w + b) = y$$

$= a\left(\sum_{i=0}^n x_i w_i + b\right)$

The diagram also includes labels for the components: 'Input' points to x , 'Weights' points to w , 'Bias' points to b , 'Model' points to the entire function f , 'Activation function' points to a , and 'Output' points to y .

A single-layer perceptron is the simplest form of a neural network. It takes multiple inputs, applies weights to them, sums them up, adds a bias, and then passes the result through an activation function to produce an output.

The core components of a perceptron are:

- **Inputs** ($x_0, \dots, x_n$): The features or data fed into the perceptron.
- **Weights** ($w_0, \dots, w_n$): Numerical values that determine the strength of the connection between inputs and the perceptron. These are adjusted during training.
- **Bias** (b): A constant value added to the weighted sum of inputs. It allows the activation function to be shifted, providing more flexibility to the model.
- **Summation** (Σ): Calculates the weighted sum of the inputs plus the bias: $z = \sum_{i=0}^n x_i w_i + b$.
- **Activation Function** ($a(z)$): A non-linear function that transforms the weighted sum into the output. It introduces non-linearity, enabling the perceptron to learn complex patterns.
- **Output** (y): The final prediction of the perceptron.

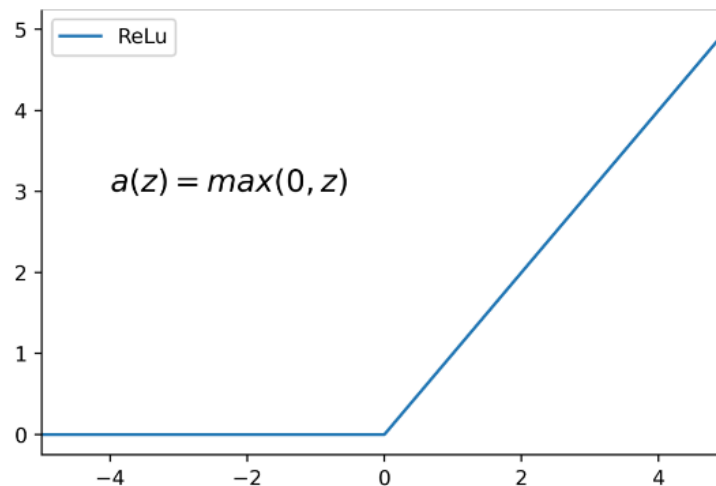
📖 The Perceptron: A Probabilistic Model - F. Rosenblatt (1958)

"The perceptron: a probabilistic model for information storage and organization in the brain."

Activation Function: Rectified Linear Unit (ReLU)

The Rectified Linear Unit (ReLU) is a popular activation function in neural networks. It is defined as:

$$a(z) = \max(0, z) = \begin{cases} z & \text{if } z > 0 \\ 0 & \text{otherwise} \end{cases} = \begin{cases} z & \text{if } x \cdot w + b > 0 \\ 0 & \text{otherwise} \end{cases}$$



This means:

- If $z > 0$, the output is z .
- If $z \leq 0$, the output is 0.

In the context of a perceptron, this translates to:

$$a(z) = \begin{cases} z & \text{if } x \cdot w + b > 0 \\ 0 & \text{otherwise} \end{cases}$$

What is an Activation Function and Why is it Needed?

An **activation function** is applied to the output of each neuron. Its primary role is to introduce **non-linearity** into the network. Without non-linear activation functions, a neural network, no matter how many layers it has, would essentially behave like a single-layer linear model. This would severely limit its ability to learn and model complex patterns in data, as most real-world relationships are non-linear.

🔗 Activation Function / ReLU even simpler!

If we only have straight lines, we can only separate data that is neatly divided by a straight line. But if we can bend and curve these lines, we can separate much more complex data distributions. Activation functions provide this "bending" capability.

ReLU's Definition and How it Works

The Rectified Linear Unit (ReLU) is one of the most widely used activation functions due to its simplicity and effectiveness. It is defined as:

$$a(z) = \max(0, z)$$

Where z is the input to the activation function

This definition means:

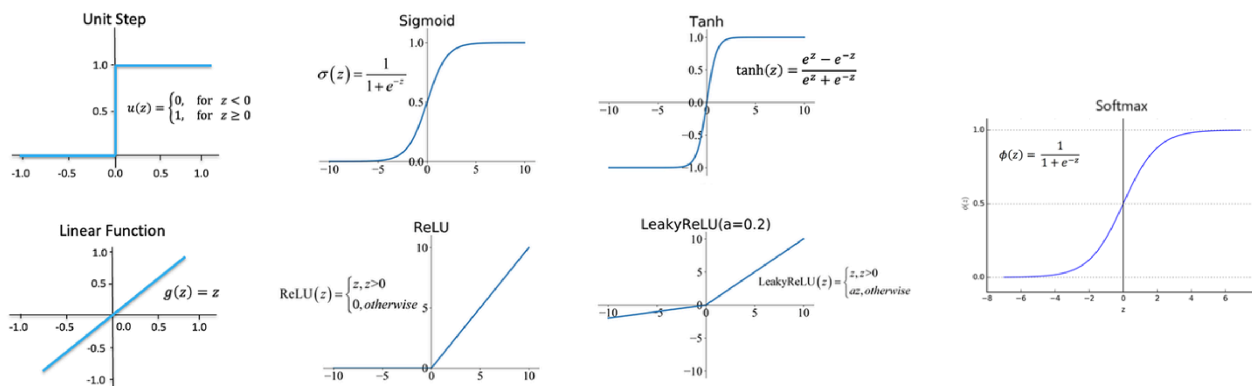
- **If the input z is greater than 0**, the output of the ReLU function is simply **z itself**. The neuron "activates" and passes on the positive value.
- **If the input z is less than or equal to 0**, the output of the ReLU function is **0**. The neuron "deactivates" or remains silent for negative inputs.

Advantages of ReLU

1. **Computational Efficiency:** ReLU is computationally very inexpensive compared to other activation functions like sigmoid or tanh, as it only involves a simple thresholding operation ($\max(0, z)$). This speeds up the training process significantly, especially in deep neural networks.
2. **Solves Vanishing Gradient Problem:** For a long time, activation functions like sigmoid and tanh suffered from the "vanishing gradient problem." In these functions, when inputs become very large or very small, their gradients (slopes) become extremely close to zero. During backpropagation, these small gradients get multiplied down through the layers, causing the updates to weights in earlier layers to become tiny, effectively stopping them from learning. ReLU, for positive inputs, has a constant gradient of 1, which helps gradients flow more easily through the network, mitigating this problem.
3. **Sparsity:** ReLU can lead to "sparse" activations. This means that a portion of the neurons will output 0 for a given input, making the network's representation sparse. This can improve computational efficiency and act as a form of regularization, potentially reducing overfitting.

[Thanks to stackoverflow](#)

Other types of activation functions



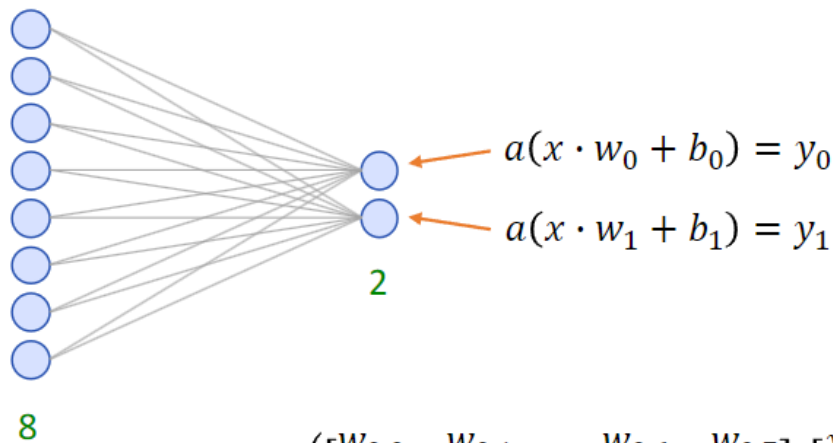
Quick Summary

- **Sigmoid:** Great for yes/no decisions. A binary choice, like is this mail spam or not (used for binary classification).
- **Tanh:** Useful for handling positive/negative distinctions, like scoring with both positive and negative values.
- **ReLU:** Ideal for ignoring minor or negative values and focusing on stronger signals, like only responding to loud sounds.
- **Leaky ReLU:** Adds a small response to negative values to prevent "dead" spots, like a dripping faucet.
- **Softmax:** Give a probability across multiple options, like choosing a favorite flavor with weighted preferences (used for multi-class classification).

[# Types of Activation Functions: Sigmoid tanh, ReLU, Softmax. Part 1](#)

Combining Perceptrons and Trainable Parameters

Neural networks are formed by combining multiple perceptrons into layers. Each neuron within a layer acts as an independent perceptron, possessing its own set of weights (w) and biases (b).



$$a \left(\begin{bmatrix} w_{0,0} & w_{0,1} & \dots & w_{0,6} & w_{0,7} \\ w_{1,0} & w_{1,1} & \dots & w_{1,6} & w_{1,7} \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \end{bmatrix} \right) = \begin{bmatrix} y_0 \\ y_1 \end{bmatrix}$$

$$a \left(\begin{bmatrix} w_{0,0} & \dots & w_{0,m} \\ \vdots & \ddots & \vdots \\ w_{n,0} & \dots & w_{n,m} \end{bmatrix} \begin{bmatrix} x_0 \\ \vdots \\ x_n \end{bmatrix} + \begin{bmatrix} b_0 \\ \vdots \\ b_n \end{bmatrix} \right) = \begin{bmatrix} y_0 \\ \vdots \\ y_n \end{bmatrix}$$

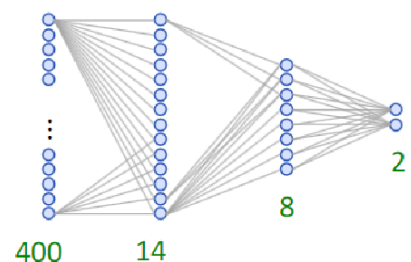
🔍 Why is it all about fast matrix multiplication?

The computation within neural networks, particularly when combining multiple perceptrons, is inherently suited for fast matrix multiplication. This is why modern deep learning frameworks heavily leverage specialized hardware like GPUs.

Trainable Parameters

The "trainable parameters" of a neural network are the weights and biases that are adjusted during the training process to minimize the model's error.

- Layers 400, 14, 8, and 2
 - => weights: $400 * 14 + 14 * 8 + 8 * 2 = 5,728$
 - => biases: $14 + 8 + 2 = 24$
 - => trainable parameter: $5,728 + 24 = 5,752$



- Trainable parameters can raise fast
 - Layers 400, 100, 40, 2 => Parameter: 44,222
 - (one model from the walkthrough)

The number of trainable parameters can increase very rapidly with more layers and neurons. For instance, a network with layer sizes 400, 100, 40, and 2 would have 44,222 parameters.

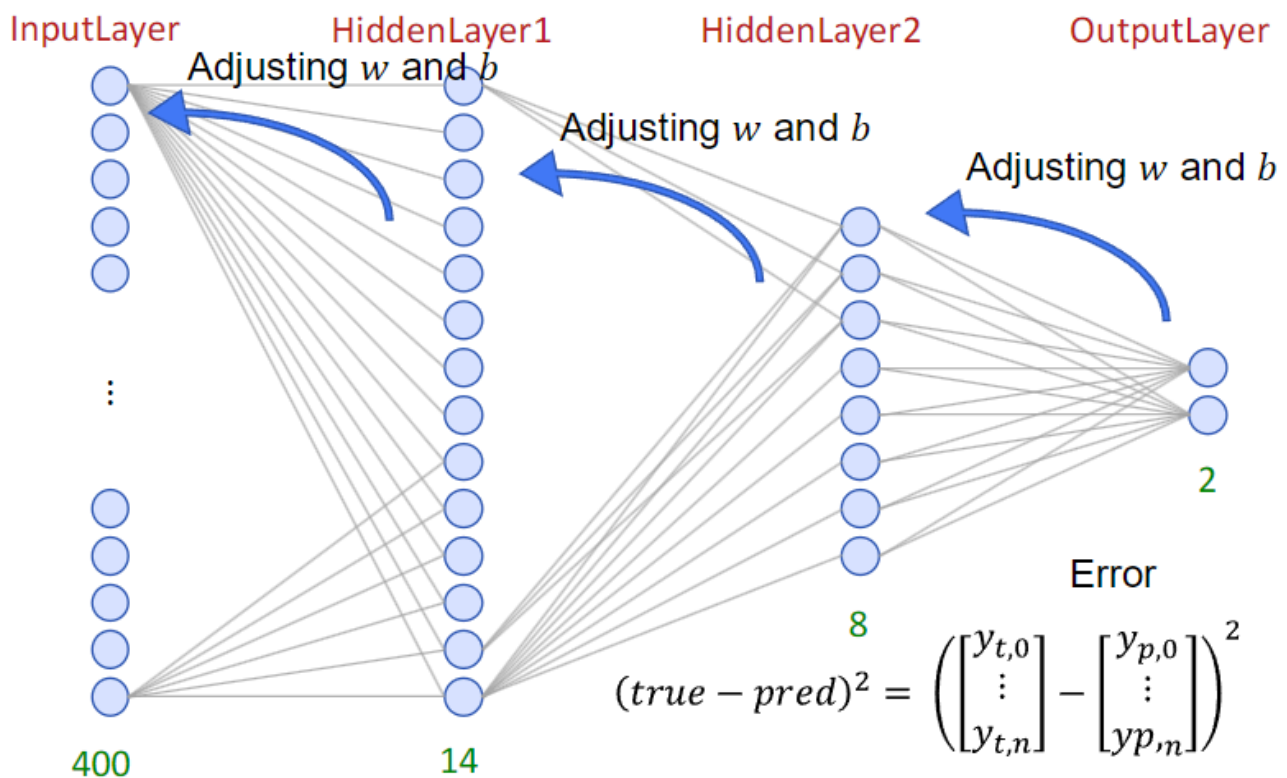
multiply weights for each layer + adding all biases = parameters

$400 * 100 + 100 * 40 + 40 * 2 + 100 + 40 + 2 = 44,222$

Trainable Parameters Calculation: calculate the number of weights and biases for a given neural network architecture.

Backpropagation

Training a neural network involves iteratively adjusting its trainable parameters (weights and biases) to minimize the difference between its predictions and the actual target values. This process can be Backpropagation.



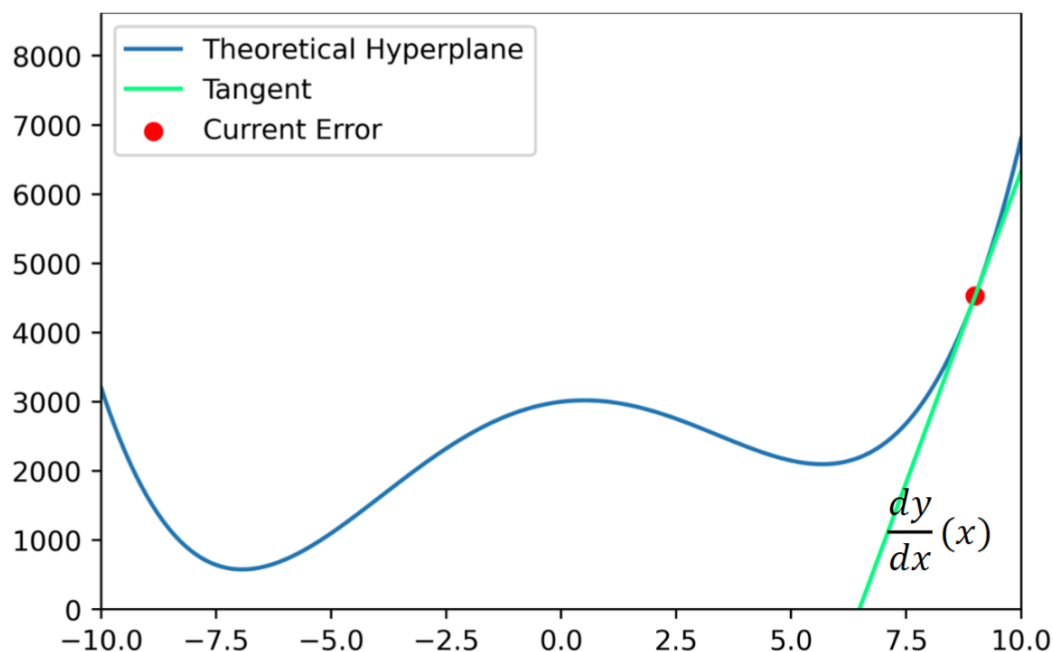
Training

1. Initialize w and b randomly
2. Determine how good the model is using a "cost function"
 - Common Cost Functions:
 - **Squared Error:** $(texttrue - textpredicted)^2$
 - **Mean Squared Error (MSE)**
 - **Mean Absolute Error (MAE)**
 - **Root Mean Squared Error (RMSE)**
3. How can we adjust w and b to be better?

Adjust Weights and Biases: The core challenge is to determine how to adjust the weights and biases to reduce the cost.

Optimization: Stochastic Gradient Descent

Stochastic Gradient Descent (SGD) is a widely used optimization algorithm for training neural networks. Its goal is to find the direction in which the cost function needs to be minimized.



The process involves calculating the gradient (slope) of the cost function with respect to each weight and bias. The gradient indicates the direction of the steepest ascent; to minimize the cost, we move in the opposite direction (down the slope).

- **Learning Rate:** The "learning rate" determines the size of the steps taken in the direction of the negative gradient. Initially, larger steps might be taken, which then decrease as the model approaches the optimal solution.
- **Batch Size:** To prevent overfitting to individual input examples and to make the training process computationally feasible, the gradient is typically calculated not for every single input, but for a "batch" of inputs. This "batch size" dictates how many input examples are processed together in each optimization step. *See Network 2 images above!*

🔗 How does Backpropagation adjust weights and biases?

Backpropagation works by calculating the error at the output layer and then propagating this error backward through the network. This process uses the chain rule of calculus to determine how much each weight and bias contributed to the overall error, allowing them to be adjusted proportionally to minimize the cost function.

🔗 Why does using squared error instead of mean error affect SGD outcomes more than choosing the optimizer?

- **Squared error amplifies larger mistakes:** Squaring the error makes big errors contribute much more to the loss than small errors. This changes the shape of the loss surface dramatically, creating steeper gradients where errors are large. As a result, the model focuses more on correcting large mistakes, which strongly influences how weights update during training.
- **Mean error treats all errors linearly:** The mean absolute error (or just mean error) weights all mistakes equally, leading to a flatter loss landscape. This can slow down learning and reduce sensitivity to outliers or big deviations.
- **Optimizer choice is secondary:** While optimizers (SGD, Adam, RMSProp, etc.) affect *how* the model updates weights (learning rate adaptation, momentum, etc.), they work on the given loss surface. If the loss function itself poorly reflects the importance of errors, even the best optimizer can only do so much.

Feature Engineering & Representation Learning

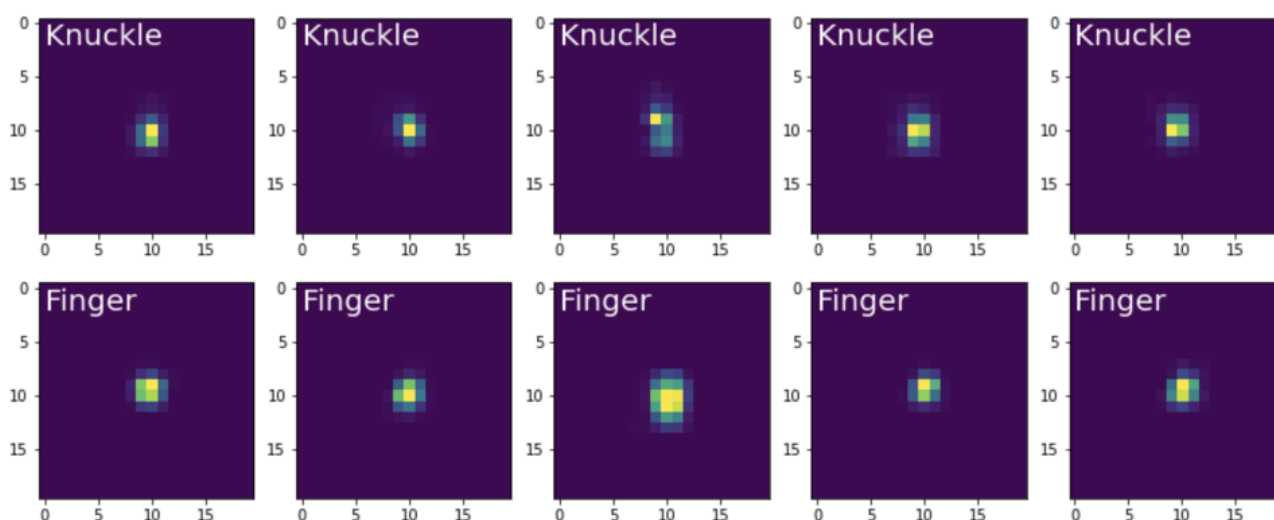
When **preparing data** for machine learning, two primary approaches exist: Feature Engineering and Representation Learning.

⚡ They are not learning strategies!

Both feature engineering and representation learning aim to provide the "best" **input** for a machine learning model. It is **data preparation and transformation** which is then crucial for the effectiveness of *both supervised and unsupervised learning* algorithms

Feature Engineering

Feature Engineering is a traditional approach to data preprocessing, particularly common in non-neural network machine learning models like Support Vector Machines (SVMs). It involves manually creating new input features from raw data based on domain expertise and prior "thinking" before the training process begins.



When analyzing images of knuckles and fingers, one might engineer features such as:

- Sum of pixels
- Minimum/Maximum pixel value
- Ellipse fitting parameters (Radius 1 & 2, Theta)
- Area of the Convex hull
- And other problem-specific features

🎨 The Art of Feature Engineering

Feature Engineering is often considered an "art" because it requires intuition, creativity, and deep understanding of the problem domain to select and transform raw data into features that are most informative for the model.

Representation Learning

In contrast to feature engineering, Representation Learning involves feeding raw data directly into the model without extensive manual preprocessing. The "hope" with representation learning, especially with neural networks, is that the model will automatically learn optimal data representations or features during the training process.

This approach requires:

- **No domain knowledge (initially):** The model is expected to discover meaningful patterns on its own.
- **No manual "thinking":** The human effort shifts from feature creation to designing appropriate network architectures and training methodologies.

Pros and Cons Comparison

Here's a comparison of Feature Engineering and Representation Learning:

Aspect	Feature Engineering	Representation Learning
Input Data	Reduced and pre-processed data	Raw data as input
Domain Knowledge	Requires significant domain knowledge and manual effort	Less initial domain knowledge required; model learns representations
Model Size	Models can be smaller	Models typically need to be larger
Suitability	More suitable for "traditional" ML models (e.g., SVM)	More suitable for Neural Network (NN) models
Training Difficulty	Easier to train smaller models	Harder to train larger, complex models
"Thinking" Burden	Human does the "thinking" (feature creation)	Model does the "thinking" (feature extraction)

❓ **What is the core difference in who or what is responsible for creating the features between Feature Engineering & Representation Learning :**

- **Feature Engineering:** Humans design and create the features.
- **Representation Learning:** The machine learning model automatically learns and discovers the features.

Data Preparation

Proper data preparation is crucial for the effective training of machine learning models. This often involves scaling and normalization techniques.

Scaling Data

The primary goal of data scaling is to transform numerical features to a common range, which helps support the activation functions within neural networks by preventing issues like vanishing or exploding gradients. Common targets for scaling include centering the mean around 0 or 0.5.

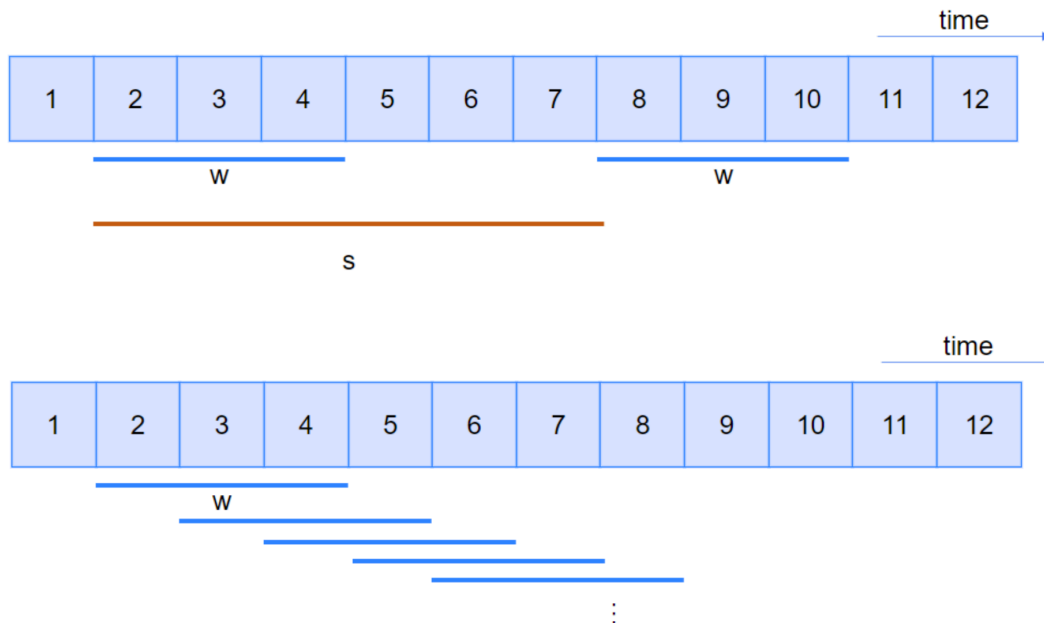
Common approaches for scaling RGB image data (typically 0-255 values):

- **Scaling to 0 to 1:** `value / 255`
- **Scaling to -1 to 1:** `value / 127.5 - 1`
Other more advanced normalization also exists!

Sliding window

This is used to transform sequential or spatial data into a format suitable for machine learning models. It involves moving a fixed-size "window" across the data, extracting segments that serve as individual input samples. This is

particularly common in areas like time series analysis, signal processing, and image processing.



w = window length

s = stride aka "step size"

🔍 The "step size" (or stride) refers to how many units the window moves forward each time.

Benefits during Training

- **Data Augmentation:** Using a step size of 1 means the sliding window moves by just one data point at a time. For example, if we have a sequence of 100 data points and a window size of 10, the number of training samples created is **$100 - 10 + 1 = 91$** . This happens because the first window covers points 1–10, the second covers 2–11, the third 3–12, and so on until the last window covers points 91–100. Each window overlaps a lot with its neighbors but is slightly shifted, giving the model many similar but unique training inputs.
- **Increased Robustness:** By exposing the model to many overlapping, slightly shifted samples, it learns to recognize patterns **regardless of small shifts or misalignments** in the data. This helps the model generalize better and be less sensitive to noise or exact positioning.

Drawbacks during Deployment

- **Computational Cost:** At inference time, processing every overlapping window is expensive because many windows share most of the same data. This leads to **redundant calculations**, slowing down prediction.
- **Inefficiency:** Since windows overlap so much, the model repeats work unnecessarily. For deployment, it's often better to use a larger step size (like equal to the window size) to reduce redundancy, even though this means fewer samples and possibly less precise detection of patterns.

🔍 Why is a step size of one good for training but bad for deployment?

- **Good for Training** because:
 - **More Training Examples:** It creates many slightly different versions of the data.
 - **Makes Model Robust:** Seeing the same pattern in slightly different positions helps the model learn to spot it no matter where it shows up.
- **Bad for Deployment** because:
 - **Too Slow & Resource Intensive:** Doing tiny step is very slow and uses a lot of compute.

- **Wasted Effort:** A lot of the work is repeated because each new window largely overlaps with the previous one.

Data Augmentation

Data augmentation is a critical technique to address the common issue of insufficient data in machine learning. It involves creating new, artificial training examples from existing ones by applying various transformations.

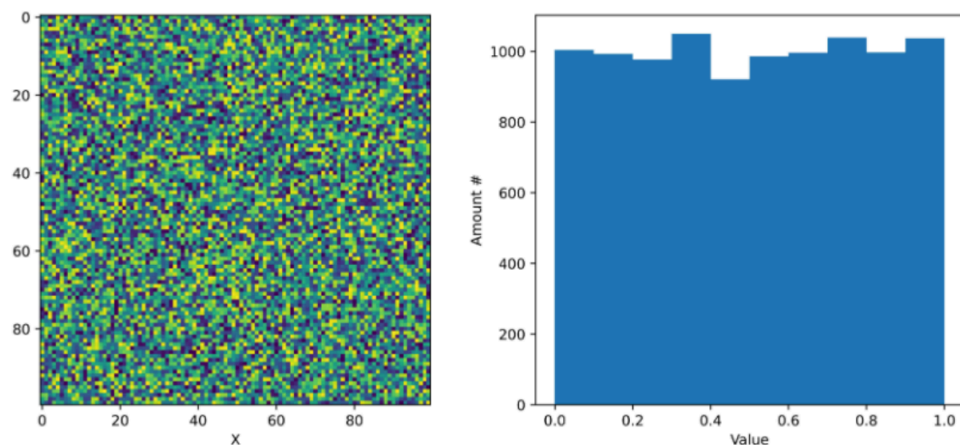
🔗 Data Augmentation Purpose - Sven Mayer

"Not enough data is a constant issue in machine learning, Data Augmentation tries to address this issue."

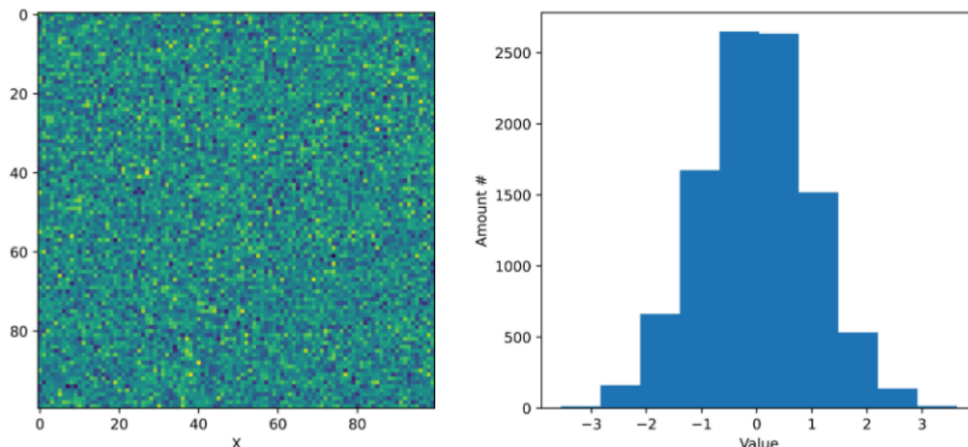
Adding Noise to Data

One common data augmentation technique is adding noise to the data. This can help make the model more robust to variations and imperfections in real-world data.

- **"True" Random Noise:** Adding completely random fluctuations to the data.

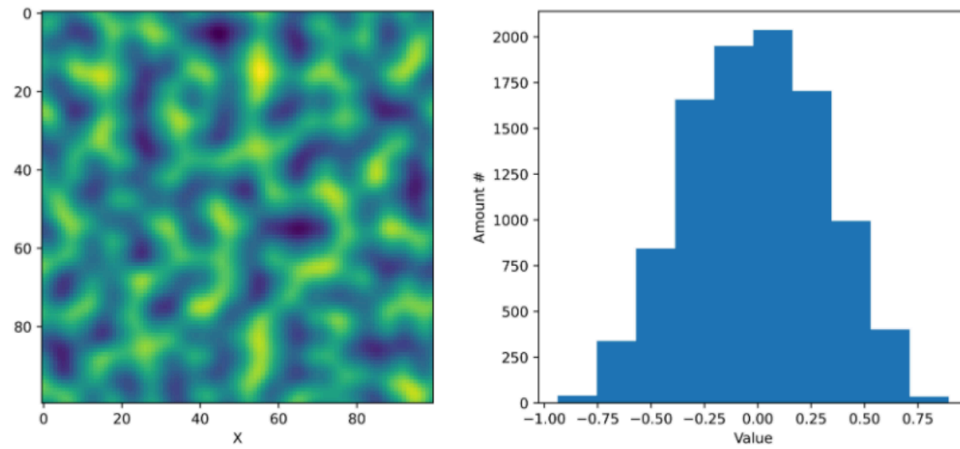


- **Normal Random Noise:** Adding noise drawn from a normal (Gaussian) distribution.



- **Perlin Noise:** A type of gradient noise often used to generate natural-looking textures and patterns. It adds a smooth curve as we do not want jittery noises everything in the world usually has more gradients than hard

edges.



General Data Augmentation Ideas

Beyond adding noise, various other transformations can be applied, especially for image data:

- **Translate:** Shifting the image horizontally or vertically.
- **Zoom/Crop:** Zooming in or out, or cropping a portion of the image.
- **Rotate:** Rotating the image by a certain angle.
- **Flip:** Mirroring the image horizontally or vertically.

Filling Methods for Unknown Information (For Images)

When dealing with images that might have missing or "unknown" information due to transformations like cropping or rotation, specific filling methods are crucial:

- **Fill with "zero":** Filling unknown regions with black pixels.
 - **Always zoom in:** Ensuring that transformations do not create empty borders by always zooming into the relevant content.
 - **Fill with other image information:** Using techniques like reflection or wrapping to fill new areas with existing image patterns.
-